Incident Report: CORP-112-309

Incident Report: Unauthorized Access & Data Destruction — CORP-112-309

Analyst: Ninu Olawale **Environment:** Live-exposed cyber range honeypot, provided by LOG(N) Pacific **Report date:** 2026-08-13

---

1. Executive Summary

An internet-exposed Windows 11 honeypot (`CORP-112-309`) running MySQL 8.0.45 was brute-forced and compromised within hours of exposure. An automated actor gained access to a MySQL database using an intentionally weak network-facing credential, enumerated and destroyed three schemas, and left a Bitcoin ransom note. A separate successful logon to the local `administrator` account occurred via RDP but showed no follow-on interactive activity. No confirmed data exfiltration or C2 traffic was observed. The device was isolated ~21 hours after exposure and remediated via credential rotation, re-hardening, and data restoration from the original seed script.

---

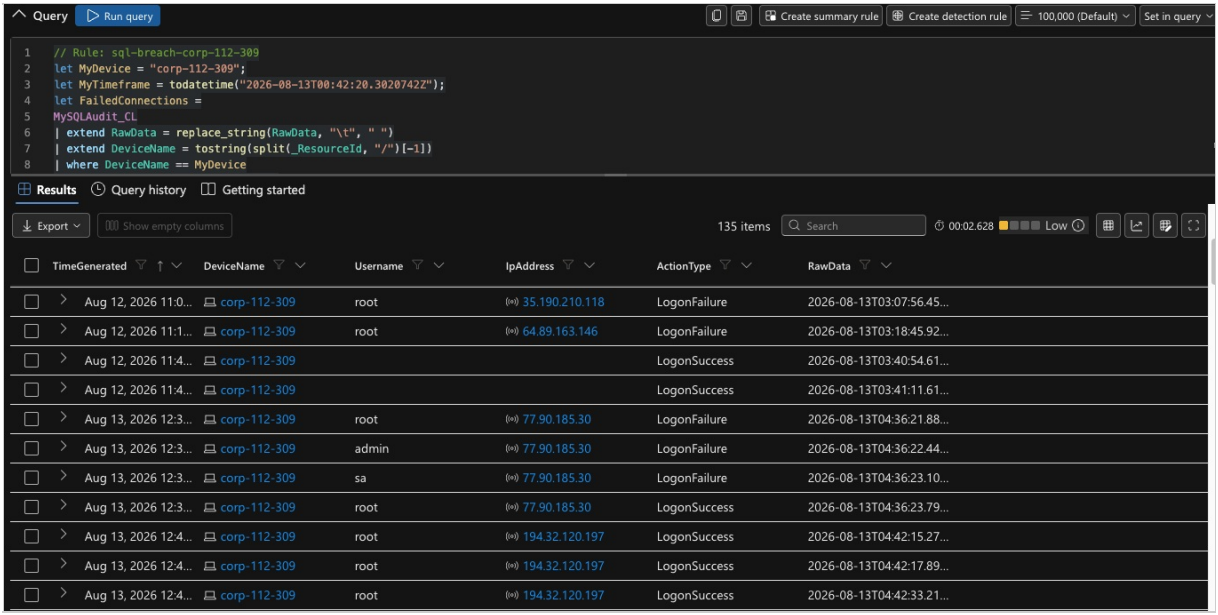
2. Incident Details

Field	Value
Detection date/time	Ongoing monitoring beginning 2026-08-13; destructive activity first observed at query-log review, ~2026-08-13 00:42–02:57 UTC
Reporter	Self-detected via scheduled analyst review of `MySQLAudit_CL` and `DeviceLogonEvents`
Classification	Unauthorized Access (T1110 Brute Force → T1078 Valid Accounts) + Data Destruction/Extortion (T1485-adjacent; note-based extortion, not confirmed encryption)
Severity	High — full database destruction on affected schemas; VM administrative account compromised
Affected assets	VM `CORP-112-309` (Windows 11 Pro); local MySQL 8.0.45 instance; schemas `lnp_corp`, `sakila`, `world`

Purpose: Standing Sentinel analytics rule — parses MySQL's raw connection log to identify successful authentications, distinguishing them from failed attempts using connection-ID correlation. This is the query that flagged the initial compromise. The original had an undeclared `MyTimeframe` variable, corrected below by adding the missing declaration.

```
// Rule: sql-breach-corp-112-309
let MyDevice = "corp-112-309";
let MyTimeframe = todatetime("2026-08-13T00:42:20.3020742Z");
let FailedConnections =
MySQLAudit_CL
| extend RawData = replace_string(RawData, "\t", " ")
| extend DeviceName = tostring(split(_ResourceId, "/")[-1])
| where DeviceName == MyDevice
| where RawData has "Access denied"
| extend ConnectionId = extract(@"^\S+\s+(\d+)\s+Connect", 1, RawData)
| distinct ConnectionId;
MySQLAudit_CL
| where TimeGenerated > MyTimeframe
| extend RawData = replace_string(RawData, "\t", " ")
| extend DeviceName = tostring(split(_ResourceId, "/")[-1])
| where DeviceName == MyDevice
| where RawData has "Connect"
| extend ConnectionId = extract(@"^\S+\s+(\d+)\s+Connect", 1, RawData)
| extend ActionType =
    case(
        RawData has "Access denied", "LogonFailure",
        ConnectionId in (FailedConnections), "Ignore",
        "LogonSuccess"
    )
| where ActionType != "Ignore"
| extend RawData = replace_string(RawData, "\t", " ")
| extend Username = replace_string(tostring(split(tostring(split(RawData,"@")[0]), " ")[-1]), "'", "")
```

```
| extend IpAddress = replace_string(tostring(split(split(RawData,"@")[1], " ")[0]), "'", "")
| project TimeGenerated, DeviceName, Username, IpAddress, ActionType, RawData
| order by TimeGenerated desc
```



sql-breach-corp-112-309 query results

3. Impact Assessment

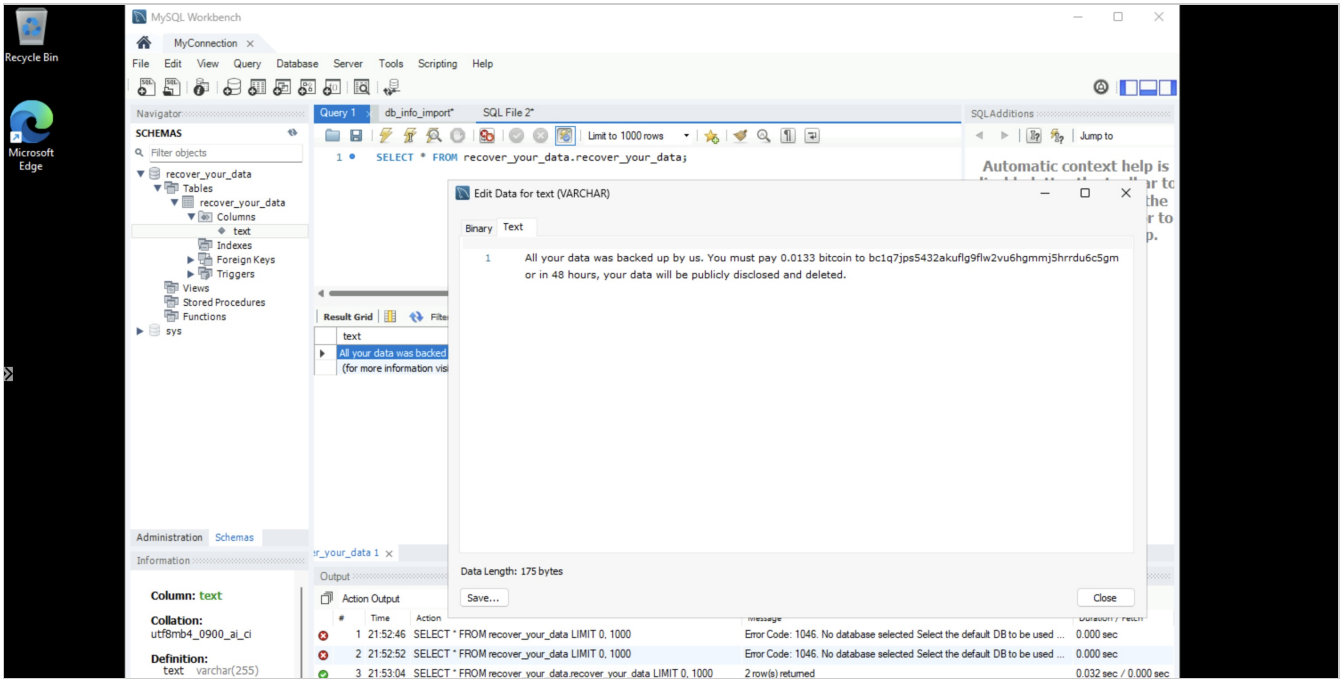
CIA aspect	Effect
Confidentiality	Attacker read schema contents (`SHOW CREATE TABLE`, `SELECT *`) prior to deletion. No confirmed exfiltration — see Evidence Section 8, `NTANetAnalytics` returned no attacker-attributable denied/allowed outbound flows carrying data.
Integrity	All tables in `lnp_corp`, `sakila`, `world` dropped and replaced with attacker-created `RECOVER_YOUR_DATA` / `RECOVER_YOUR_DATA_info` marker tables.
Availability	Full loss of original schema contents until manual restoration.
Business impact	No production data or customer-facing systems were affected. Impact assessed as contained to the affected host and database instance.

4. Indicators of Compromise

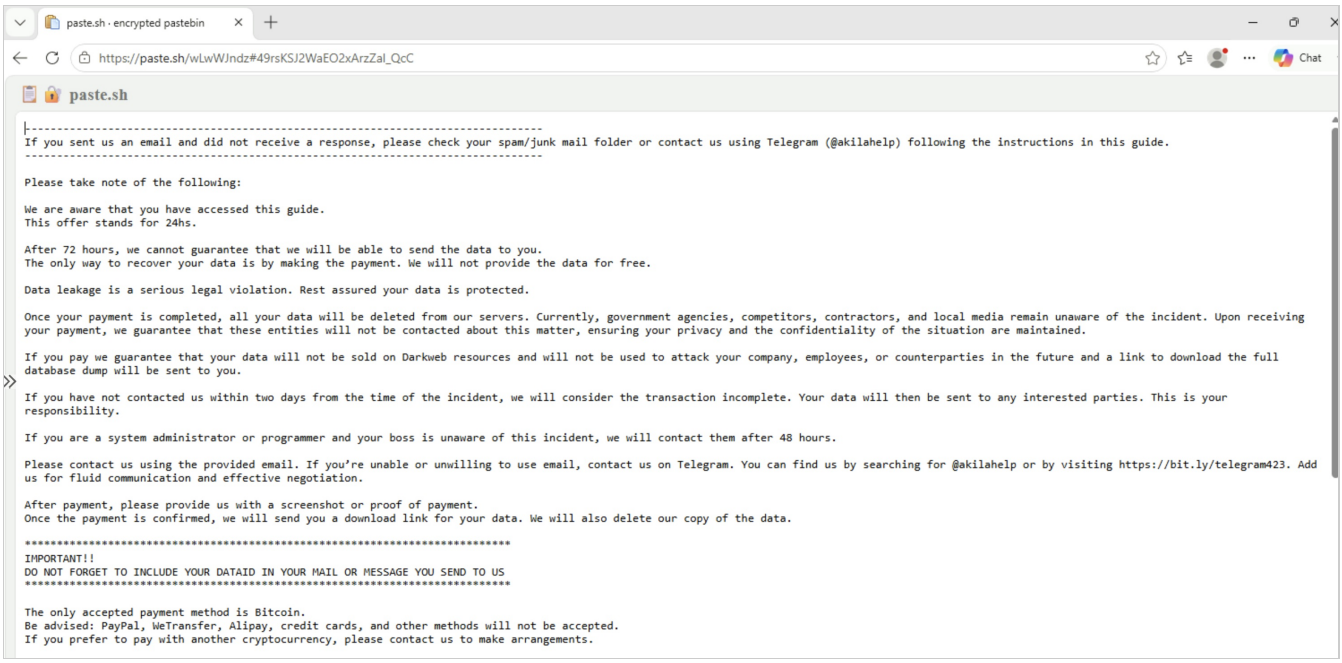
Type	Value	Source
BTC address	`bc1q7jps5432akuflg9flw2vu6hgmmj5hrrdu6c5gm`	MySQLAudit_CL (Query)
Contact email	`ak+27i5n@onionmail.org`	MySQLAudit_CL (Query)
Reference URL	`hxxps://bit[.]ly/22mysql`	MySQLAudit_CL (Query)
Secondary contact channel	Telegram handle `@akilahelp`, reachable via `hxxps://bit[.]ly/telegram423`	Ransom landing page (paste.sh)
DATAID	`27I5N`	MySQLAudit_CL (Query)
Ransom artifact table	`RECOVER_YOUR_DATA`, `RECOVER_YOUR_DATA_info` (created in `lnp_corp`, `sakila`, `world`)	MySQLAudit_CL (Query)
MySQL auth source IPs (13 distinct)	`77.90.185.30`, `64.89.163.165`, `77.90.185.21`, `64.89.163.94`, `64.89.163.141`, `64.89.163.154`, `64.89.163.139`, `213.209.159.115`, `194.32.120.197`, `35.195.204.188`, `103.234.99.115`, `64.89.163.146`,	MySQLAudit_CL (Auth)

	`35.190.210.118`, `64.89.163.148`	
IP: destructive session #1 (initial dump+drop, all 3 schemas)	`194.32.120.197`	MySQLAudit_CL (Auth × Query, time-correlated)
IP: destructive session #2 (final ransom note + privilege change)	`64.89.163.165`	MySQLAudit_CL (Auth × Query, time-correlated)
RDP source IP — confirmed successful `administrator` logon	`59.15.116.99`	DeviceLogonEvents
Compromised account (MySQL)	`root@'%'` (intentional bait credential, password `root`)	MySQLAudit_CL
Compromised account (Windows)	`administrator` (intentional bait credential)	DeviceLogonEvents

Screenshot — ransom note as left in MySQL (`RECOVER_YOUR_DATA` table contents):



Screenshot — extortion landing page reached via the reference URL in the ransom note:



Note: this page was accessed by the analyst for evidence-gathering purposes only, navigating from the URL embedded in the attacker's ransom note — no payment or contact was made. The page provides operational detail on the extortion scheme (72-hour data-sale threshold, Bitcoin-only payment, Telegram as a secondary contact channel) consistent with a templated, automated extortion campaign rather than bespoke targeting of this host.

5. Timeline (UTC)

Time	Event	Source
2026-08-11 20:10	VM `CORP-112-309` created	Azure resource metadata
2026-08-12 21:58:38	First MySQL auth event recorded	MySQLAudit_CL (Auth)
2026-08-13 00:42:20	NSG change to Allow-All-Inbound	Analyst record
2026-08-13 00:36:21–00:36:23	Brute-force success on bait `root@'%'` from `77.90.185.30`	MySQLAudit_CL (Auth)
2026-08-13 00:42:15–00:43:35	Session from `194.32.120.197`: enumerates all tables in `lnp_corp`, `sakila`, `world` (`SHOW CREATE TABLE`), drops all tables, inserts first-pass “backup” marker rows	MySQLAudit_CL (Query)
2026-08-13 02:56:32–02:57:00	Session from `64.89.163.165`: drops `world`, `sakila`, `recover_your_data` databases; recreates `RECOVER_YOUR_DATA`; inserts full ransom note (BTC address, email, DATAID); modifies `root@'%'` privileges (`REVOKE ALL`, `GRANT SHUTDOWN`)	MySQLAudit_CL (Query)
2026-08-13 07:46–11:26	Four recurring `CREATE DATABASE RECOVER_YOUR_DATA` statements — consistent with repeat automated scans re-checking the box	MySQLAudit_CL (Query)
2026-08-13 11:46:03	Successful `administrator` RDP logon from `59.15.116.99`; zero process activity attributed to the account over the following 6 hours	DeviceLogonEvents, DeviceProcessEvents
2026-08-13 21:58:22	Device isolated via Microsoft Defender	Analyst-recorded, MDE Portal action

6. Root Cause / Attack Vector

- **Initial access vector:** Internet-facing MySQL (TCP 3306) and RDP (TCP 3389), reachable following an NSG rule change to Allow-All-Inbound, combined with weak credentials (`root@'%'` / `root`; weak `administrator` password).
- **Access method:** Direct credential brute-force / guessing against default-style usernames (`root`, `sa`, `admin`) from 13 distinct IPs — consistent with mass internet scanning rather than reconnaissance targeted at this specific host.
- **Not determined from available logs:** exact tooling/fingerprint used by the attacker (MySQL general query log does not capture client user-agent or tool banners); whether the RDP and MySQL brute-force campaigns originated from the same actor/infrastructure (no IP overlap observed between the two sets).

7. Response Actions

Containment - Device isolated via Microsoft Defender for Endpoint (full isolation), 2026-08-13 21:58:22Z. - Second Investigation Package captured for before/after forensic comparison.

Eradication - NSG hardened (reverted from Allow-All-Inbound). - Windows Firewall re-enabled. - Full Windows Defender malware scan executed. - `administrator` account deleted; `guest` account disabled. - Primary local account credentials rotated to strong/unique values. - MySQL closed to public internet; bait `root@'%'` account remediated (password rotated / account removed).

Recovery - `lnp_corp` schema restored by re-running the original seed import (`db_info_import.sql`). - **Limitation to note:** this restores the original seed dataset, not a true point-in-time backup taken immediately before compromise — flagged as a process gap (see Section 9).

Purpose: Verify no further logons occurred on this host after isolation, confirming containment held.

```
let MyDevice = "corp-112-309";
let IsolationTime = todatetime("2026-08-13T21:58:22.9317788Z");
DeviceLogonEvents
| where DeviceName == MyDevice
| where TimeGenerated > IsolationTime
| project TimeGenerated, AccountName, RemoteIP, ActionType, LogonType
| order by TimeGenerated asc
```

Result: no rows returned. No logons occurred on this host after isolation.

---

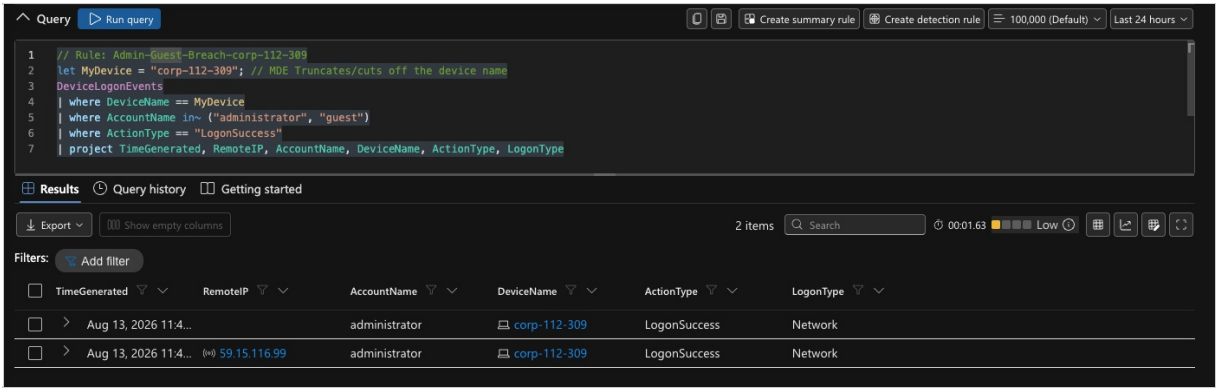
8. Evidence

- `MySQLAudit_CL-Auth_Logs_csv.csv` — 104 auth events, 13 source IPs, success/failure breakdown
- `MySQLAudit_CL-Queries.csv` — 752 query events including full ransom-note payload and schema destruction sequence
- `MySQLAudit_CL-Log_On_Events.csv` (DeviceLogonEvents export) — confirmed `administrator` RDP logon
- `Pre_Breach_MDE_Investigation_Package.zip` — baseline system state, active connections, local group membership, prior to exposure
- `Post_Breach_MDE_Investigation_Package.zip` — post-isolation state; `mysqld.exe` process confirmed running from expected install path (`C:\Program Files\MySQL\MySQL Server 8.0\bin\mysqld.exe`) — no evidence of binary replacement or malicious persistence found in `Processes.csv` or Prefetch listing
- `NTANetAnalytics` query — 5 denied outbound flows, all benign NTP (UDP/123) from the host itself; no attacker-attributable denied/allowed outbound activity found

Detection rules deployed prior to exposure:

Purpose: Standing Sentinel analytics rule — alerts on any successful logon to the bait `administrator` or `guest` accounts on this host.

```
// Rule: Admin-Guest-Breach-corp-112-309
let MyDevice = "corp-112-309"; // MDE Truncates/cuts off the device name
DeviceLogonEvents
| where DeviceName == MyDevice
| where AccountName in~ ("administrator", "guest")
| where ActionType == "LogonSuccess"
| project TimeGenerated, RemoteIP, AccountName, DeviceName, ActionType, LogonType
```



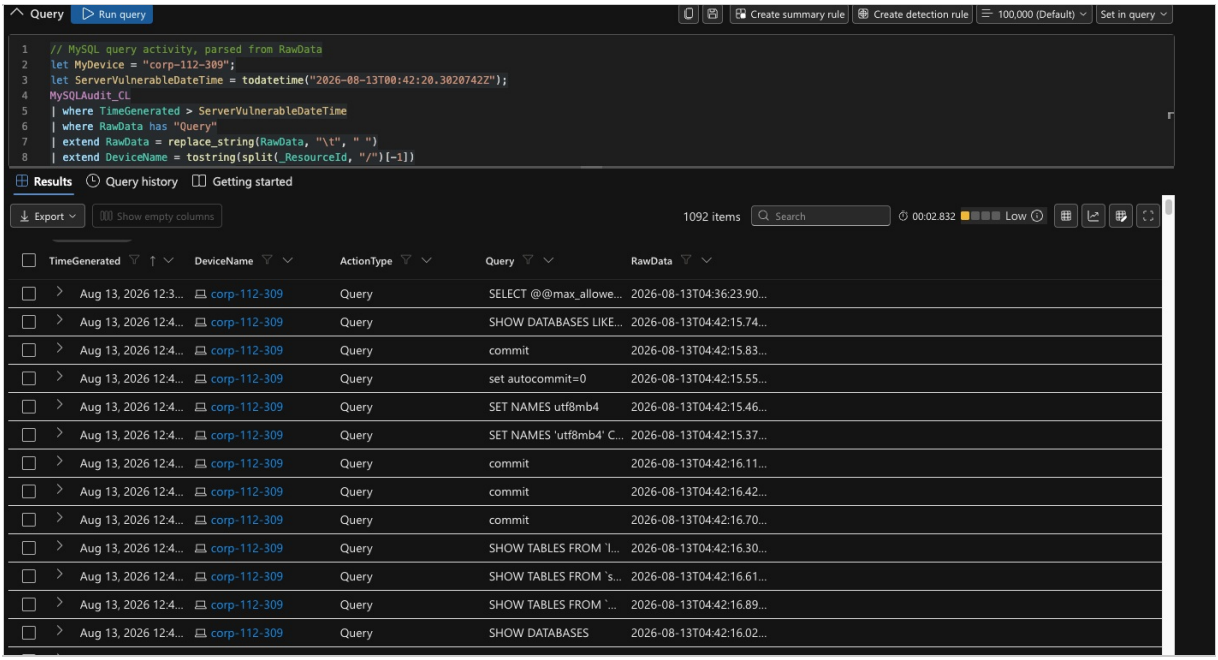
Admin-Guest-Breach-corp-112-309 rule results

The second standing rule, `sql-breach-corp-112-309`, is documented in Incident Details (Section 2) alongside the earliest-signal discussion, since it's the query that flagged the initial compromise.

Detection queries used during active monitoring:

Purpose: Pull all MySQL query-level activity (not just auth events) for this host from the point of exposure onward, to see what commands were run after a successful connection.

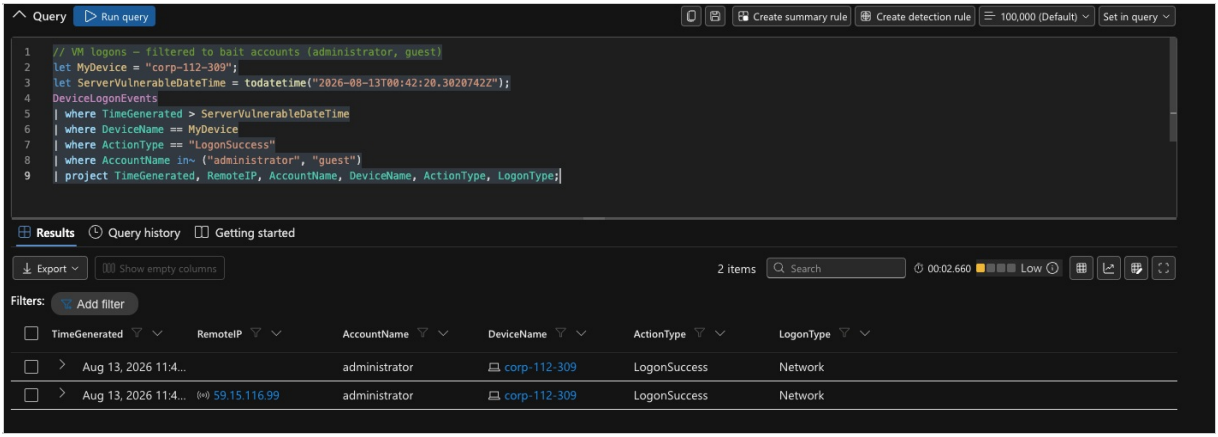
```
// MySQL query activity, parsed from RawData
let MyDevice = "corp-112-309";
let ServerVulnerableDateTime = todatetime("2026-08-13T00:42:20.3020742Z");
MySQLAudit_CL
| where TimeGenerated > ServerVulnerableDateTime
| where RawData has "Query"
| extend RawData = replace_string(RawData, "\t", " ")
| extend DeviceName = tostring(split(_ResourceId, "/")[-1])
| where DeviceName == MyDevice
| extend ActionType = "Query"
| extend Query = split(RawData, "Query")[1]
| project TimeGenerated, DeviceName, ActionType, Query, RawData
| order by TimeGenerated desc
```



MySQL query activity results

Purpose: Check for successful logons to the bait `administrator`/`guest` accounts from the point of exposure onward, run ad hoc during active monitoring rather than waiting on the standing analytics rule.

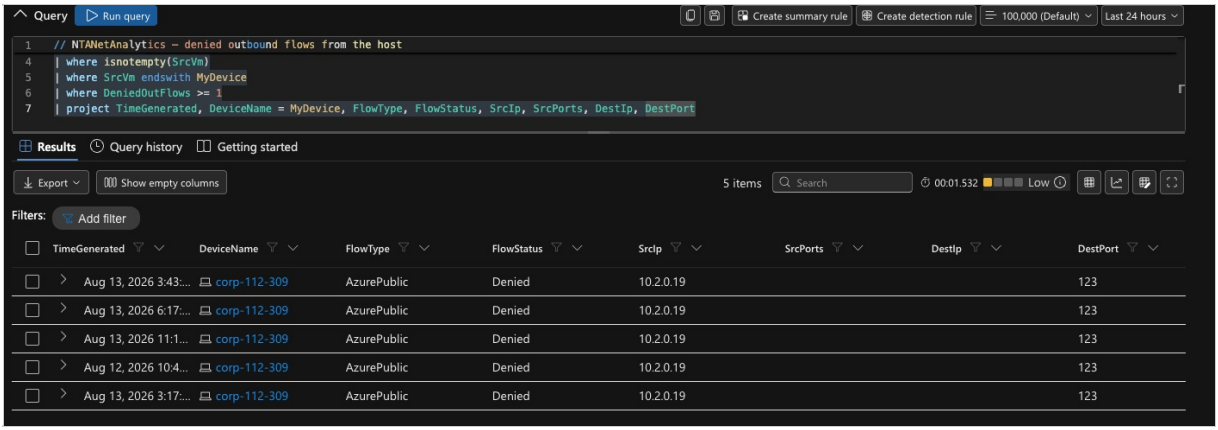
```
// VM logons — filtered to bait accounts (administrator, guest)
let MyDevice = "corp-112-309";
let ServerVulnerableDateTime = todatetime("2026-08-13T00:42:20.3020742Z");
DeviceLogonEvents
| where TimeGenerated > ServerVulnerableDateTime
| where DeviceName == MyDevice
| where ActionType == "LogonSuccess"
| where AccountName in~ ("administrator", "guest")
| project TimeGenerated, RemoteIP, AccountName, DeviceName, ActionType, LogonType;
```



VM logon check — bait accounts, filtered to exposure window

Purpose: Check for outbound network flows from this host that were denied by the tenant egress policy, to look for attempted command-and-control or exfiltration traffic.

```
// NTANetAnalytics — denied outbound flows from the host
let MyDevice = "corp-112-309";
NTANetAnalytics
| where isnotempty(SrcVm)
| where SrcVm endswith MyDevice
| where DeniedOutFlows >= 1
| project TimeGenerated, DeviceName = MyDevice, FlowType, FlowStatus, SrcIp, SrcPorts, DestIp, DestPort
```



NTANetAnalytics denied outbound flows

Purpose: ad hoc hunt checking whether any process activity on the host was attributed to the `administrator` account during or after its RDP session, to determine whether the successful logon was followed by hands-on-keyboard activity.

```
let MyDevice = "corp-112-309";
let LogonTime = todatetime("2026-08-13T11:46:03Z");
DeviceProcessEvents
| where DeviceName == MyDevice
| where TimeGenerated between (LogonTime - 5m .. LogonTime + 6h)
| where AccountName =~ "administrator" or InitiatingProcessAccountName =~ "administrator"
| project TimeGenerated, AccountName, FileName, ProcessCommandLine
```

Result: no rows returned.

---

9. Lessons Learned / Recommendations

1. **(High) Build proactive detection for this attack pattern.** No analytics rule existed to detect ransom-table creation (`CREATE TABLE ... RECOVER_YOUR_DATA`, extortion-note inserts) as it happened; the compromise was identified through scheduled manual log review rather than automated alerting. A targeted detection rule would reduce time-to-detect from hours to near-real-time.
2. **(Medium) Preserve host-based firewall logging.** Local firewall logging (`pfirewall.log`) was unavailable for this incident because the Windows Firewall was disabled prior to compromise, removing a normally free source of connection-level evidence. Firewall logging should remain enabled independent of firewall blocking policy.
3. **(Medium) Validate recovery against true backups, not seed data.** Recovery in this incident restored the original seed dataset rather than a point-in-time backup taken immediately prior to compromise. A scheduled backup process is recommended so recovery reflects actual pre-incident state rather than initial provisioning state.
4. **(Low) Reassess exposure-window duration.** The majority of high-value findings (full database compromise chain, RDP credential validation) occurred within the first 12 hours of exposure; the remaining exposure time produced additional volume but no new attacker technique, suggesting shorter monitoring windows can be sufficient for similar internet-facing assets.